

Question

Write a Java program to check whether brackets are balanced using a stack.

Steps for Execution

1. Save the code as `ValidParentheses.java`.
2. Compile the program.
3. Run `ValidParentheses`.
4. Enter a string of brackets.

Algorithm

1. Create an empty stack.
2. Push each opening bracket onto the stack.
3. For a closing bracket, check the most recent opening bracket.
4. Return false if a pair does not match.
5. Return true if all pairs match and the stack is empty.

UML Class Diagram

```
+-----+
|      ValidParentheses      |
+-----+
| + isValid(s: String): boolean |
| + main(args: String[]): void  |
+-----+

    | uses
    +---- Stack<Character>
    +---- Scanner
```

CODE: SRC

ValidParentheses.java

```

import java.util.*;

public class ValidParentheses {

    public static boolean isValid(String s) {

        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {

            // Add opening brackets to stack
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            }

            // Check closing brackets
            else if (ch == ')' || ch == '}' || ch == ']') {

                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {

                    return false;
                }
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter parentheses: ");
        String input = sc.nextLine();
    }
}

```

```
        if (isValid(input)) {
            System.out.println("Valid");
        } else {
            System.out.println("Not valid");
        }

        sc.close();
    }
}
```

Compilation and Execution

```
javac ValidParentheses.java
java ValidParentheses
```

Output

```
Enter parentheses: ({[]})
Valid
```